S3-FIFO

FIFO Queues Are All You Need for Cache Eviction

A simple and efficient cache eviction algorithm

Juncheng Yang et al. · SOSP 2023

Operating Systems · 10-minute group presentation

01 BACKGROUND

Member 1 · 3 minutes

02 ALGORITHM

Member 2 · 4 minutes

03 EVALUATION

Member 3 · 3 minutes

Presenters: Member 1 / Member 2 / Member 3

What Is Cache Eviction?

Fast access

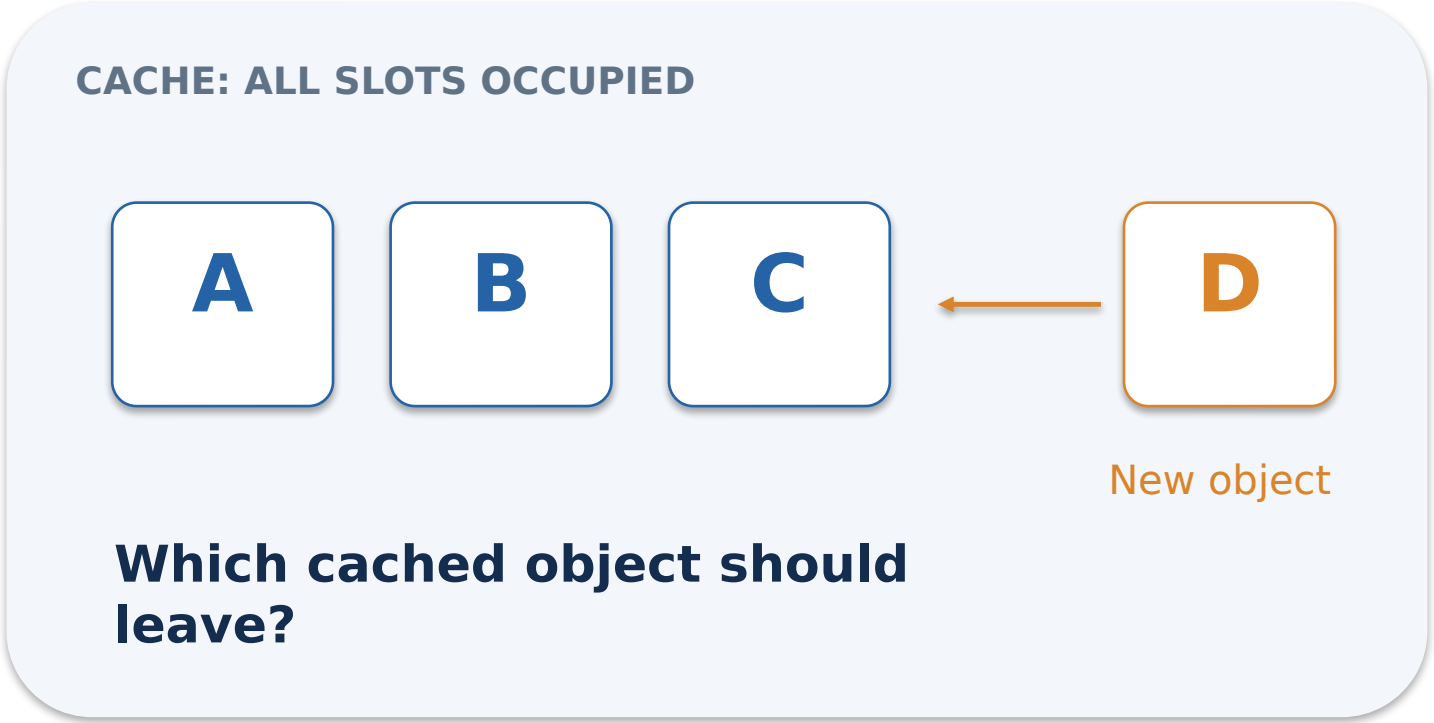
A cache keeps data for reuse.

Limited space

A full cache needs room for new data.

Eviction

Choose an object to remove.



Cache eviction = choosing what to remove when space is needed.

Source: Yang et al., SOSP 2023, §§1–2 (PDF pp. 1–2).

Traditional Algorithms: FIFO vs LRU

FIFO

First In, First Out

- Evict the earliest inserted object
- Simple and fast
- May remove useful data

LRU

Least Recently Used

- Evict the least recently accessed object
- Update object order on cache hits
- Updates add concurrency overhead

The challenge: keep FIFO’s simplicity while improving cache efficiency.

Source: Yang et al., SOSP 2023, §2.2 (PDF p. 3).

Research Motivation: One-Hit Wonders

One-hit wonders

Objects used only once before eviction.

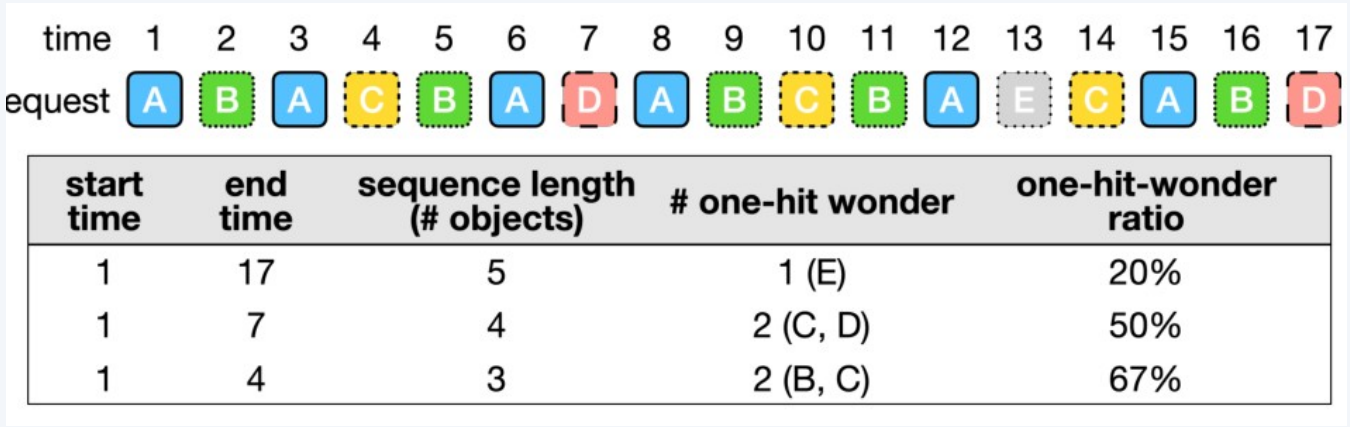
Wasted cache space

No reuse benefit within the short window.

Quick demotion

Remove less useful objects early.

SHORTER WINDOW → MORE ONE-HIT WONDERS



start time	end time	sequence length (# objects)	# one-hit wonder	one-hit-wonder ratio
1	17	5	1 (E)	20%
1	7	4	2 (C, D)	50%
1	4	3	2 (B, C)	67%

Original Figure 1 · Yang et al., SOSP 2023

Can simple FIFO queues also deliver high cache efficiency?

Source: original Figure 1 and §3, Yang et al., SOSP 2023 (PDF pp. 3–5).

Overview of S3-FIFO

S **Small**

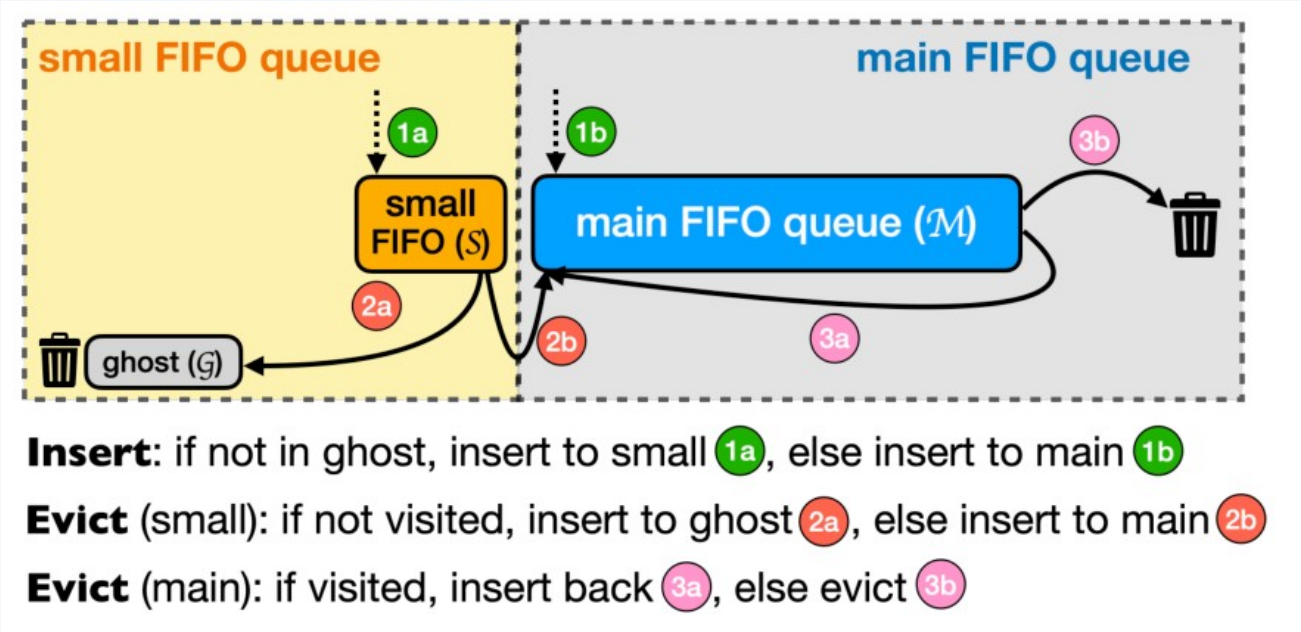
Filter newly inserted objects.

M **Main**

Retain objects with reuse.

G **Ghost**

Remember IDs, not cached data.



Original Figure 5 · Yang et al., SOSP 2023

Three FIFO queues work together: filter, protect, remember.

Source: original Figure 5 and §4.1, Yang et al., SOSP 2023 (PDF p. 5).

Three Queues and Their Roles

Cached data: Small + Main = 100% of cache capacity



SMALL / S

10%

New-object testing area

Filters low-reuse objects

MAIN / M

90%

Objects with demonstrated reuse

Retains useful cached data

GHOST / G

IDs only

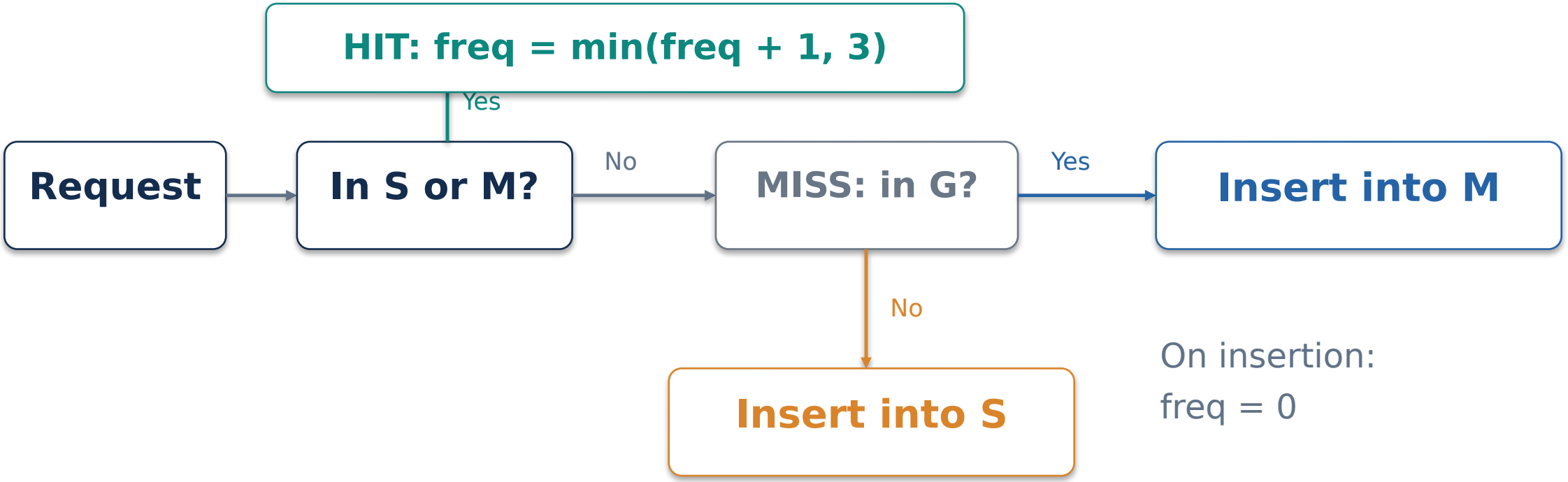
Recently evicted object IDs

No cached payload

Ghost entries are metadata; they do not hold the actual cached data.

Source: §4.1 and Figure 5, Yang et al., SOSP 2023 (PDF p. 5).

Cache Hit and Cache Miss



On a cache hit: update the counter; keep the queue order unchanged.

Source: Algorithm 1, lines 1–13, Yang et al., SOSP 2023 (PDF p. 6). Simplified flowchart.

Eviction Process

When space is needed: evict from S if S ≥ 10% of capacity; otherwise evict from M.

SMALL QUEUE / S

Oldest object: freq > 1?

Yes → **Move to M**

No → **Remove data; ID → G**

MAIN QUEUE / M

Oldest object: freq > 0?

Yes → **Reinsert; freq − 1**

No → **Evict object**

Quick demotion in S + second chances in M; fewer queue updates on hits.

Source: Algorithm 1, lines 14–40, Yang et al., SOSP 2023 (PDF p. 6). Simplified flowcharts.

Experimental Setup

6,594
REAL-WORLD CACHE TRACES

14
DATASETS

3
WORKLOAD CATEGORIES

Block	6 datasets	MSR, FIU, CloudPhysics, Systor, Tencent, Alibaba
Key-value	3 datasets	Twitter, Social Network, Meta KV
Object / CDN	5 datasets	CDN 1, CDN 2, Tencent Photo, Wikimedia, Meta CDN

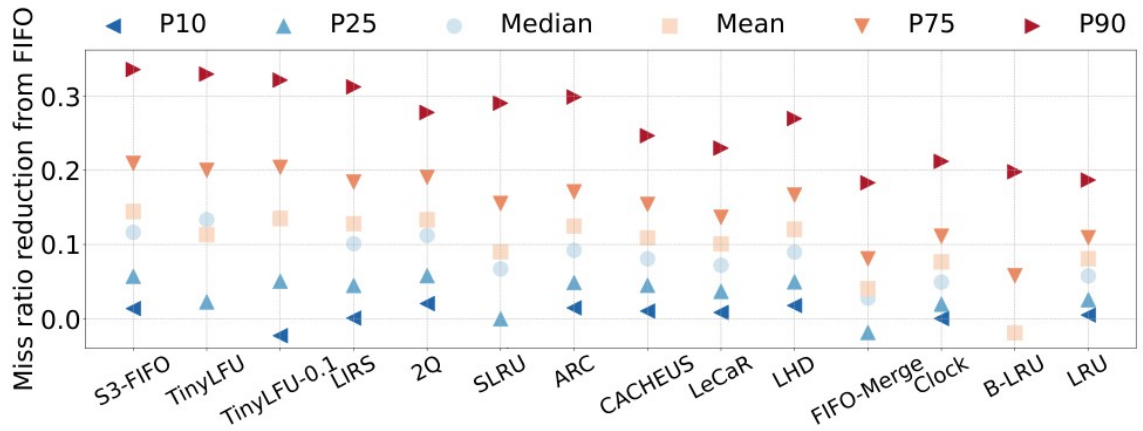
Baselines: FIFO, LRU and other policies · Metrics: Miss Ratio & Throughput

Reported experiments by the paper’s authors; not reproduced by our group.

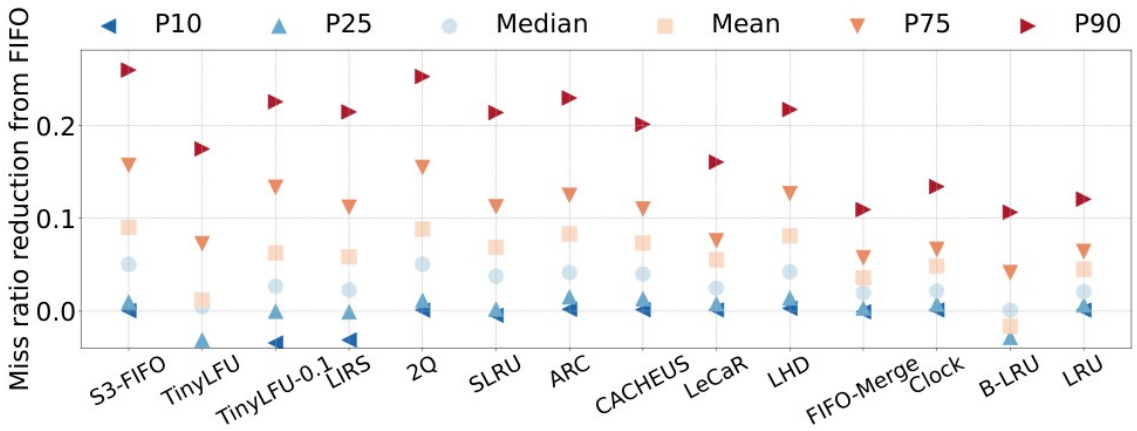
Source: Table 1 (category summary) and §5.1, Yang et al., SOSP 2023 (PDF p. 7).

Results: Cache Efficiency

Miss Ratio: lower is better. Figure 6 shows reduction relative to FIFO: higher is better.



(a) Large cache size, 10% trace footprint



(b) Small cache size, 0.1% trace footprint

Original Figure 6 · All axes, algorithm labels and percentile legends retained.

10 / 14

Lowest mean miss ratio in 10 of 14 datasets

Larger cache setting · Robust efficiency across many workloads

Source: Figure 6; §§5.1–5.2 (PDF pp. 8–9). Dataset-level 10/14 result: Figure 7 and §5.2. Authors' results.

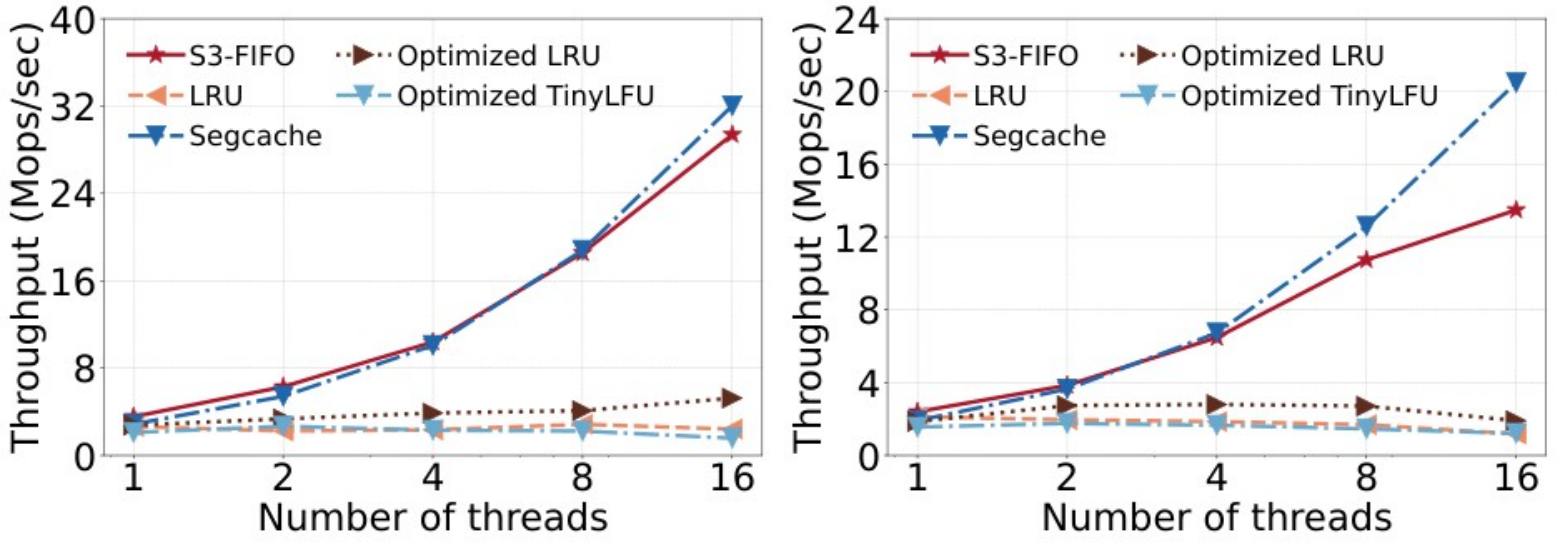
Results: Throughput & Scalability

THROUGHPUT

Requests handled per second

>6×

vs optimized LRU
at 16 threads



Original Figure 8 · Synthetic Zipf workload (α = 1.0)

Fewer queue updates on cache hits support higher throughput and scalability.

Source: original Figure 8 and §5.3, Yang et al., SOSP 2023 (PDF p. 10). Authors' results.

Limitations & Conclusion

LIMITATION: ACCESS PATTERNS STILL MATTER

An object may leave S before its second request → an extra cache miss.

S Filter

Quick demotion

M Protect

Retain useful data

G Remember

IDs without payload

Three simple FIFO queues → low miss ratios + better scalability.

Source: §§5.2 and 8, Yang et al., SOSP 2023 (PDF pp. 9, 14).

References

PRIMARY PAPER

Yang, J., Zhang, Y., Qiu, Z., Yue, Y., & Rashmi, K. V. (2023). FIFO Queues Are All You Need for Cache Eviction. SOSP ’23.

DOI

https://doi.org/10.1145/3600006.3613147

OFFICIAL CODE & DATA

https://github.com/TheSys-lab/sosp23-s3fifo

Figures 1, 5, 6 and 8 reproduced from the provided paper. Mechanism diagrams redrawn from Algorithm 1.